

1. Podstawowe komponenty aplikacji Android i ich cykle życia:

- **Activity (Aktywność):**
 - Pojedynczy ekran interfejsu użytkownika, z którym użytkownik może wchodzić w interakcje.
 - Każda aktywność ma swój cykl życia (stany: `created`, `started`, `resumed`, `paused`, `stopped`, `destroyed`), zarządzany przez system.
 - Metody cyklu życia (np. `onCreate()`, `onStart()`, `onResume()`, `onPause()`, `onStop()`, `onDestroy()`, `onRestart()`) są wywoływane w zależności od stanu aktywności.
- **Fragment:**
 - Modułarna część interfejsu użytkownika, która może być osadzona w aktywności.
 - Fragmenty mają własne cykle życia, które są powiązane z cyklem życia ich hostującej aktywności.
 - Umożliwiają tworzenie elastycznych interfejsów dla różnych rozmiarów ekranów.
- **Service (Usługa):**
 - Komponent działający w tle, bez interfejsu użytkownika.
 - Służy do wykonywania długotrwałych operacji (np. odtwarzanie muzyki, pobieranie danych) bez blokowania interfejsu.
 - Może działać jako `started` (uruchomiona przez `startService()`) lub `bound` (powiązana z innymi komponentami przez `bindService()`).
- **Broadcast Receiver (Odbiorca Rozgłoszeń):**
 - Komponent reagujący na rozgłoszenia systemowe lub zdefiniowane przez aplikację (np. niski poziom baterii, połączenie sieciowe).
 - Nie ma interfejsu użytkownika.
- **Content Provider (Dostawca Zawartości):**
 - Udostępnia dane aplikacji innym aplikacjom w bezpieczny i ustrukturyzowany sposób.
 - Abstrahuje źródło danych (np. bazę danych SQLite, pliki, sieć) i umożliwia dostęp poprzez jednolity interfejs.

2. Struktura i zarządzanie interfejsem graficznym (GUI) w Androidzie:

- **Definicja GUI:** Interfejs użytkownika jest definiowany głównie w plikach XML (dla separacji od logiki biznesowej) lub programistycznie w kodzie Java/Kotlin (dla dynamicznego tworzenia lub modyfikacji).
- **Zasoby (Resources):** Elementy takie jak układy (layouts), ciągi znaków (strings), kolory, obrazy są przechowywane w folderze `res` i referencjonowane w XML lub kodzie, co ułatwia lokalizację i adaptację do różnych urządzeń.
- **Podstawowe elementy GUI:**
 - **View:** Podstawowy blok konstrukcyjny interfejsu, reprezentujący prostokątny obszar ekranu, który reaguje na interakcje. Przykłady: `TextView`, `Button`, `ImageView`.

- **ViewGroup**: Kontener dla innych **View** i **ViewGroup**, służący do organizacji elementów interfejsu (np. **LinearLayout**, **RelativeLayout**, **ConstraintLayout**, **FrameLayout**).
- **Android Studio**: Zapewnia edytor GUI z widokami Design i Blueprint, drzewem komponentów oraz panelami atrybutów, ułatwiając projektowanie interfejsu.

3. Architektura systemu Android:

- **Warstwy (od góry do dołu)**:
 - **Applications (Aplikacje)**: Zainstalowane aplikacje użytkownika oraz aplikacje systemowe.
 - **Application Framework (Framework Aplikacji)**: Zestaw API (Activity Manager, Window Manager, Content Providers, View System, Package Manager, Telephony Manager, Location Manager, Notification Manager) umożliwiający deweloperom tworzenie aplikacji i dostęp do funkcji systemowych.
 - **Android Runtime (ART) / Dalvik VM**: Środowisko wykonawcze dla aplikacji Androidowych (patrz punkt 4).
 - **Native Libraries (Biblioteki Natywne)**: Biblioteki C/C++ (np. SQLite, WebKit, OpenGL ES, Media Framework), które dostarczają podstawowe funkcjonalności.
 - **Hardware Abstraction Layer (HAL)**: Interfejs między frameworkiem Androida a sterownikami sprzętowymi, pozwalający na niezależność od konkretnego sprzętu.
 - **Linux Kernel (Jądro Linuksa)**: Podstawa systemu, odpowiedzialna za zarządzanie procesami, pamięcią, siecią, sterownikami sprzętowymi itp.

4. Wirtualne maszyny Androida (Dalvik i ART):

- **Dalvik Virtual Machine (DVM)**:
 - Starsza wirtualna maszyna Androida, używająca kompilacji Just-In-Time (JIT).
 - Jest maszyną stosową (**stack machine**), co oznacza, że operacje są wykonywane na stosie.
 - Kompilowała kod bytecode'u **.dex** (Dalvik Executable) do kodu maszynowego w trakcie działania aplikacji.
- **Android Runtime (ART)**:
 - Nowsza wirtualna maszyna, zastąpiła Dalvika od Androida 5.0 (Lollipop).
 - Używa kompilacji Ahead-Of-Time (AOT), co oznacza, że kod aplikacji jest kompilowany do kodu maszynowego podczas instalacji.
 - Jest maszyną rejestrową (**register machine**), co często prowadzi do wydajniejszej i szybszej egzekucji kodu.
 - Oferuje lepszą wydajność, dłuższy czas pracy na baterii i ulepszone zarządzanie pamięcią w porównaniu do Dalvika.

5. Struktura systemu plików Android i metody dostępu do danych:

- **Struktura plików:** System plików Android jest oparty na Linuksie. Każda aplikacja ma swój własny katalog danych (`/data/data/<nazwa_pakietu>`), do którego inne aplikacje nie mają dostępu domyślnie.
- **Metody dostępu do danych:**
 - **Pamięć wewnętrzna (Internal Storage):** Prywatna pamięć aplikacji, niedostępna dla innych aplikacji. Idealna dla wrażliwych danych.
 - **Pamięć zewnętrzna (External Storage):** Dostępna dla wszystkich aplikacji (za zgodą użytkownika). Może być publiczna (widoczna dla użytkownika) lub prywatna (specyficzna dla aplikacji, usuwana po odinstalowaniu).
 - **SQLite Database:** Wbudowana, lekka baza danych relacyjnych. Aplikacje często używają jej do przechowywania ustrukturyzowanych danych lokalnie.
 - **SharedPreferences:** Prosty mechanizm do przechowywania małych kolekcji par klucz-wartość (np. ustawienia użytkownika).
 - **Content Providers:** Jak wspomniano w punkcie 1, służą do bezpiecznego udostępniania danych między aplikacjami.

6. Mechanizmy komunikacji międzyaplikacyjnej (IPC) w Androidzie:

- **Intents (Zamiary):**
 - **Explicit Intents (Jawne Zamiary):** Określają konkretny komponent docelowy (np. `startActivity(new Intent(this, TargetActivity.class))`). Używane do uruchamiania komponentów w tej samej aplikacji lub w znanych aplikacjach.
 - **Implicit Intents (Niejawne Zamiary):** Deklarują akcję do wykonania i typ danych, a system Android znajduje odpowiedni komponent do jej obsługi (np. otworenie strony internetowej, wysłanie e-maila).
 - **Intent Filters (Filtry Zamiarów):** Deklarowane w `AndroidManifest.xml`, określają, jakie typy zamiarów może obsłużyć dany komponent.
- **Komunikacja z usługami powiązanymi (Bound Service Communication):**
 - **Binder:** Podstawowy mechanizm IPC w Androidzie. Umożliwia procesom wywoływanie metod w innych procesach.
 - **Messenger:** Klasa opakowująca `Binder` i ułatwiająca komunikację opartą na wiadomościach między procesami.
 - **AIDL (Android Interface Definition Language):** Język definicji interfejsu służący do definiowania interfejsów programistycznych, które mogą być używane do komunikacji między procesami. System generuje kod Java na podstawie pliku AIDL.

7. Programowanie natywne w Androidzie (NDK, JNI, ABI):

- **Android Native Development Kit (NDK):**
 - Zestaw narzędzi pozwalający deweloperom na pisanie części aplikacji Android w językach C i C++.

- Przydatny w przypadku operacji wymagających wysokiej wydajności (np. gry, przetwarzanie sygnałów, algorytmy kryptograficzne) lub gdy istnieje potrzeba ponownego wykorzystania istniejącego kodu C/C++.
- **Java Native Interface (JNI):**
 - Framework, który umożliwia kodowi Java (lub Kotlin) wywoływanie kodu natywnego (C/C++) i na odwrót.
 - Służy jako pomost między maszynami wirtualnymi (ART) a bibliotekami natywnymi.
- **Application Binary Interface (ABI):**
 - Definiuje, w jaki sposób kod maszynowy aplikacji powinien wchodzić w interakcje z systemem (np. jak są wywoływane funkcje, jak dane są przekazywane).
 - Android obsługuje różne architektury procesorów (np. **armeabi-v7a**, **arm64-v8a**, **x86**, **x86_64**), a NDK pozwala na kompilację kodu natywnego dla konkretnych ABI.

8. Model bezpieczeństwa Androida:

- **Konta użytkowników:** Android jest systemem wieloużytkownikowym, co oznacza, że każdy użytkownik ma oddzielne i chronione środowisko.
- **Izolacja aplikacji (App Sandbox):**
 - Każda aplikacja uruchamiana jest w oddzielnym procesie Linuxa z własnym unikalnym identyfikatorem użytkownika (UID).
 - Procesy aplikacji są izolowane od siebie, a dostęp do zasobów systemowych i danych innych aplikacji jest ściśle kontrolowany poprzez system uprawnień.
 - Aplikacje nie mają domyślnie uprawnień do dostępu do danych innych aplikacji, chyba że wyraźnie zostaną im przyznane przez użytkownika lub system.
- **System uprawnień:** Aplikacje muszą jawnie prosić o uprawnienia do korzystania z wrażliwych zasobów (np. aparat, lokalizacja, kontakty, dostęp do pamięci). Uprawnienia są klasyfikowane jako "normalne" (przyznawane automatycznie) lub "niebezpieczne" (wymagające zgody użytkownika).
- **Root Access (Dostęp do roota):**
 - **Czym jest root:** Uzyskanie uprawnień superużytkownika (root) w systemie Android, co daje pełną kontrolę nad urządzeniem. Obejmuje to modyfikację plików systemowych, instalację niestandardowych ROM-ów i dostęp do funkcji normalnie niedostępnych.
 - **Consequences (Konsekwencje):**
 - **Utrata gwarancji:** Zazwyczaj unieważnia gwarancję producenta.
 - **Problemy z SafetyNet:** Zrootowane urządzenia mogą nie przejść kontroli SafetyNet Hardware Attestation (ponieważ łańcuch certyfikatów sprzętowych zostanie przerwany), co powoduje, że usługi Google (np. Google Pay) i niektóre aplikacje (np. bankowe, strumieniowe z DRM Widevine) uznają urządzenie za niezabezpieczone i mogą odmówić działania lub oferować ograniczoną funkcjonalność (np. Netflix tylko w 720p).

- **Ryzyko bezpieczeństwa:** Aplikacje z dostępem do roota mogą potencjalnie wyciekać dane, wprowadzać złośliwe oprogramowanie lub nawet uszkodzić sprzęt urządzenia. Wymaga to dużej ostrożności i wiedzy technicznej.
- **Skomplikowany proces:** Zazwyczaj wymaga odblokowania bootloadera i flashowania niestandardowego recovery (np. TWRP), co jest procesem technicznym.